PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Exception Handling, File I/O, Modules and Packages

Exceptions

Lekha A
Computer Applications

- An exception is an unexpected event that occurs during program execution.
- These events can disrupt the normal flow of the program and potentially cause it to crash.
- To handle exceptions effectively, Python provides a mechanism called exception handling, which allows programmers to anticipate and deal with potential errors gracefully.

- They represent errors or exceptional conditions that arise when a script is running.
- When an exceptional event occurs in Python, an object known as an "exception" is raised.
- This object contains information about the error, such as its type and, often, additional data about what went wrong.

- Exception handling involves two main components
    - Raising exceptions
    - Handling exceptions.

- Raising Exceptions

  - Raising an exception is the process of notifying the program that an error has occurred.

  - This is done using the raise keyword followed by an exception object.

- Handling Exceptions
  - Handling exceptions involves using the try and except blocks.
  - The try block contains the code that needs to be monitored for potential errors.
  - The except block contains the code that should be executed when an exception is raised.

- There are built-in exceptions in Python that are raised when corresponding errors occur.
- It can be viewed using
    - print(dir(locals()['__builtins__']))

Try .. except block

- The try...except block is used to handle exceptions in Python.
- try:
-     # code that may cause exception
- except:
-     # code to run when exception occurs

Try .. except block

-
```
try:
    numerator = 10
    denominator = 0

    result = numerator/denominator

    print(result)
except:
    print("Error: Denominator cannot be 0.")
```

```
Error: Denominator cannot be 0.
```

Try .. except block

- To handle the exception, put the code, **result = numerator/denominator** inside the try block.
- Now when an exception occurs, the rest of the code inside the try block is skipped.

Try .. except block

- The except block catches the exception and statements inside the except block are executed.
- If none of the statements in the try block generates an exception, the except block is skipped.

Try .. except block

- ```
  try:
      even_numbers = [2,4,6,8]
      print(even_numbers[5])

  except ZeroDivisionError:
      print("Denominator cannot be 0.")

  except IndexError:
      print("Index Out of Bound.")
  ```

  ```
  Index Out of Bound.
  ```
  ```

Try .. except block

- ```
x = 5
y = "hello"
try:
    z = x + y
except TypeError:
    print("Error: cannot add an int and a str")
```

```
Error: cannot add an int and a str
```
```

Try .. else block

- To run a certain block of code if the code block inside try runs without any errors.
- For these cases, use the optional else keyword with the try statement.

Try .. else block


```
try:
    num = int(input("Enter a number: "))
    assert num % 2 == 0
except:
    print("Not an even number!")
else:
    reciprocal = 1/num
    print(reciprocal)
```

```
Enter a number:  10
0.1
```

Try .. finally block

- The finally block is always executed no matter whether there is an exception or not.
- The finally block is optional.
- For each try block, there can be only one finally block.

Try .. finally block

```
try:
    numerator = 10
    denominator = 0

    result = numerator/denominator

    print(result)
except:
    print("Error: Denominator cannot be 0.")

finally:
    print("This is finally block.")
```

```
Error: Denominator cannot be 0.
This is finally block.
```

Recap

- Exceptions
- Try Except
- Try Else
- Try Finally

THANK YOU

Lekha A

Computer Applications